FastAPI Reference

WebSockets

When defining WebSockets, you normally declare a parameter of type `WebSocket` and with it you can read data from the client and send data to it.

Read more about it in the FastAPI docs for WebSockets

It is provided directly by Starlette, but you can import it from `fastapi`:

```
from fastapi import WebSocket
```

Tip

When you want to define dependencies that should be compatible with both HTTP and WebSockets, you can define a parameter that takes an `HTTPConnection` instead of a `Request` or a `WebSocket`.

class fastapi.WebSocket

```
WebSocket(scope, receive, send)
```

Bases: `HTTPConnection`

Source code in `starlette/websockets.py`

```
def __init__(self, scope: Scope, receive: Receive, send: Send) -> None:
    super().__init__(scope)
    assert scope["type"] == "websocket"
    self._receive = receive
    self._send = send
    self.client_state = WebSocketState.CONNECTING
    self.application_state = WebSocketState.CONNECTING
```

attr scope `instance-attribute`

```
scope = scope
```

attr app `property`

```
app
```

attr url `property`

```
url
```

attr base_url `property`

Ask AI

```
base_url
```

attr headers `property`

```
headers
```

attr query_params `property`

```
query_params
```

attr path_params `property`

```
path_params
```

attr cookies `property`

```
cookies
```

attr client `property`

```
client
```

attr state `property`

```
state
```

attr client_state `instance-attribute`

```
client_state = CONNECTING
```

attr application_state `instance-attribute`

```
application_state = CONNECTING
```

meth url_for

```
url_for(name, /, **path_params)
```

Source code in `starlette/requests.py`

```
def url_for(self, name: str, /, **path_params: Any) -> URL:
    url_path_provider: Router | Starlette | None = self.scope.get("router") or
self.scope.get("app")
    if url_path_provider is None:
        raise RuntimeError("The `url_for` method can only be used inside a Starlette application
or with a router.")
    url_path = url_path_provider.url_path_for(name, **path_params)
    return url_path.make_absolute_url(base_url=self.base_url)
```

meth receive `async`

```
receive()
```

Receive ASGI websocket messages, ensuring valid state transitions.

Source code in `starlette/websockets.py`

```
35  async def receive(self) -> Message:
36      """
37      Receive ASGI websocket messages, ensuring valid state transitions.
38      """
39      if self.client_state == WebSocketState.CONNECTING:
40          message = await self._receive()
41          message_type = message["type"]
42          if message_type != "websocket.connect":
43              raise RuntimeError(f'Expected ASGI message "websocket.connect", but got
44  {message_type!r}')
45          self.client_state = WebSocketState.CONNECTED
46          return message
47      elif self.client_state == WebSocketState.CONNECTED:
48          message = await self._receive()
49          message_type = message["type"]
50          if message_type not in {"websocket.receive", "websocket.disconnect"}:
51              raise RuntimeError(
52                  f'Expected ASGI message "websocket.receive" or "websocket.disconnect", but got
53  {message_type!r}'
54              )
55          if message_type == "websocket.disconnect":
56              self.client_state = WebSocketState.DISCONNECTED
57          return message
    else:
        raise RuntimeError('Cannot call "receive" once a disconnect message has been received.')
```

meth send `async`

```
send(message)
```

Send ASGI websocket messages, ensuring valid state transitions.

Source code in `starlette/websockets.py`

```
async def send(self, message: Message) -> None:
    """
    Send ASGI websocket messages, ensuring valid state transitions.
    """
    if self.application_state == WebSocketState.CONNECTING:
        message_type = message["type"]
        if message_type not in {"websocket.accept", "websocket.close",
"websocket.http.response.start"}:
            raise RuntimeError(
                'Expected ASGI message "websocket.accept", "websocket.close" or
"websocket.http.response.start", '
                f"but got {message_type!r}"
            )
        if message_type == "websocket.close":
            self.application_state = WebSocketState.DISCONNECTED
        elif message_type == "websocket.http.response.start":
            self.application_state = WebSocketState.RESPONSE
        else:
            self.application_state = WebSocketState.CONNECTED
        await self._send(message)
    elif self.application_state == WebSocketState.CONNECTED:
        message_type = message["type"]
        if message_type not in {"websocket.send", "websocket.close"}:
            raise RuntimeError(
                f'Expected ASGI message "websocket.send" or "websocket.close", but got
{message_type!r}'
            )
        if message_type == "websocket.close":
            self.application_state = WebSocketState.DISCONNECTED
        try:
            await self._send(message)
        except OSError:
            self.application_state = WebSocketState.DISCONNECTED
            raise WebSocketDisconnect(code=1006)
    elif self.application_state == WebSocketState.RESPONSE:
        message_type = message["type"]
        if message_type != "websocket.http.response.body":
            raise RuntimeError(f'Expected ASGI message "websocket.http.response.body", but got
{message_type!r}')
        if not message.get("more_body", False):
            self.application_state = WebSocketState.DISCONNECTED
        await self._send(message)
    else:
        raise RuntimeError('Cannot call "send" once a close message has been sent.')
```

meth accept `async`

```
accept(subprotocol=None, headers=None)
```

Source code in `starlette/websockets.py`

```
 async def accept(
     self,
     subprotocol: str | None = None,
     headers: Iterable[tuple[bytes, bytes]] | None = None,
 ) -> None:
     headers = headers or []

     if self.client_state == WebSocketState.CONNECTING:  # pragma: no branch
         # If we haven't yet seen the 'connect' message, then wait for it first.
         await self.receive()
     await self.send({"type": "websocket.accept", "subprotocol": subprotocol, "headers": headers})
```

meth receive_text `async`

```
receive_text()
```

Source code in `starlette/websockets.py`

```
116 async def receive_text(self) -> str:
117     if self.application_state != WebSocketState.CONNECTED:
118         raise RuntimeError('WebSocket is not connected. Need to call "accept" first.')
119     message = await self.receive()
120     self._raise_on_disconnect(message)
121     return cast(str, message["text"])
```

meth receive_bytes `async`

```
receive_bytes()
```

Source code in `starlette/websockets.py`

```
123 async def receive_bytes(self) -> bytes:
124     if self.application_state != WebSocketState.CONNECTED:
125         raise RuntimeError('WebSocket is not connected. Need to call "accept" first.')
126     message = await self.receive()
127     self._raise_on_disconnect(message)
128     return cast(bytes, message["bytes"])
```

meth receive_json `async`

```
receive_json(mode='text')
```

Source code in `starlette/websockets.py`

```
130 async def receive_json(self, mode: str = "text") -> Any:
131     if mode not in {"text", "binary"}:
132         raise RuntimeError('The "mode" argument should be "text" or "binary".')
133     if self.application_state != WebSocketState.CONNECTED:
134         raise RuntimeError('WebSocket is not connected. Need to call "accept" first.')
135     message = await self.receive()
136     self._raise_on_disconnect(message)
137
138     if mode == "text":
139         text = message["text"]
140     else:
141         text = message["bytes"].decode("utf-8")
142     return json.loads(text)
```

meth iter_text `async`

```
iter_text()
```

Source code in `starlette/websockets.py`

```
144 async def iter_text(self) -> AsyncIterator[str]:
145     try:
146         while True:
147             yield await self.receive_text()
148     except WebSocketDisconnect:
149         pass
```

meth iter_bytes `async`

```
iter_bytes()
```

Source code in `starlette/websockets.py`

```
151 async def iter_bytes(self) -> AsyncIterator[bytes]:
152     try:
153         while True:
154             yield await self.receive_bytes()
155     except WebSocketDisconnect:
156         pass
```

meth iter_json `async`

```
iter_json()
```

Source code in `starlette/websockets.py`

```
158 async def iter_json(self) -> AsyncIterator[Any]:
159     try:
160         while True:
161             yield await self.receive_json()
162     except WebSocketDisconnect:
163         pass
```

meth send_text `async`

```
send_text(data)
```

Source code in `starlette/websockets.py`

```
165 async def send_text(self, data: str) -> None:
166     await self.send({"type": "websocket.send", "text": data})
```

meth send_bytes `async`

```
send_bytes(data)
```

Source code in `starlette/websockets.py`

```
async def send_bytes(self, data: bytes) -> None:
    await self.send({"type": "websocket.send", "bytes": data})
```

meth send_json `async`

```
send_json(data, mode='text')
```

Source code in `starlette/websockets.py`

```
async def send_json(self, data: Any, mode: str = "text") -> None:
    if mode not in {"text", "binary"}:
        raise RuntimeError('The "mode" argument should be "text" or "binary".')
    text = json.dumps(data, separators=(",", ":"), ensure_ascii=False)
    if mode == "text":
        await self.send({"type": "websocket.send", "text": text})
    else:
        await self.send({"type": "websocket.send", "bytes": text.encode("utf-8")})
```

meth close `async`

```
close(code=1000, reason=None)
```

Source code in `starlette/websockets.py`

```
async def close(self, code: int = 1000, reason: str | None = None) -> None:
    await self.send({"type": "websocket.close", "code": code, "reason": reason or ""})
```

WebSockets - additional classes

Additional classes for handling WebSockets.

Provided directly by Starlette, but you can import it from `fastapi`:

```
from fastapi.websockets import WebSocketDisconnect, WebSocketState
```

class fastapi.websockets.WebSocketDisconnect

```
WebSocketDisconnect(code=1000, reason=None)
```

Bases: `Exception`

Source code in `starlette/websockets.py`

```
def __init__(self, code: int = 1000, reason: str | None = None) -> None:
    self.code = code
    self.reason = reason or ""
```

attr code `instance-attribute`

```
code = code
```

attr reason `instance-attribute`

```
reason = reason or ''
```

When a client disconnects, a `WebSocketDisconnect` exception is raised, you can catch it.

You can import it directly form `fastapi`:

```
from fastapi import WebSocketDisconnect
```

Read more about it in the FastAPI docs for WebSockets

class fastapi.websockets.WebSocketState

Bases: `Enum`

attr CONNECTING `class-attribute` `instance-attribute`

```
CONNECTING = 0
```

attr CONNECTED `class-attribute` `instance-attribute`

```
CONNECTED = 1
```

attr DISCONNECTED `class-attribute` `instance-attribute`

```
DISCONNECTED = 2
```

attr RESPONSE `class-attribute` `instance-attribute`

```
RESPONSE = 3
```

`WebSocketState` is an enumeration of the possible states of a WebSocket connection.